



DOCUMENTATION CODEGEN POLICY

Версия: 1.0.0

Дата: 2026-06-13

Статус: Активная политика

Автор: Koda (NLP-Core-Team)



НАЗНАЧЕНИЕ

Этот документ фиксирует правила использования документации AI/codegen workflows для генерации кода.

Primary Purpose: Обеспечить использование canonical contracts как основы для code generation.



CODEGEN WORKFLOW

Порядок чтения перед codegen:

- | | |
|---|---------------------|
| 1. SUMMARY_DOCS/INDEX.md | # Overview |
| 2. SUMMARY_DOCS/MANIFEST.json | # Document list |
| 3. SUMMARY_DOCS/appendix/AI_ENTRYPOINTS.md | # AI instructions |
| 4. SUMMARY_DOCS/playbooks/codegen-playbook.md | # Codegen guide |
| 5. relevant contracts/ | # Specifications |
| 6. relevant state/ | # Configuration |
| 7. relevant topology/ | # Architecture maps |

Context building:

1. **Load contracts** — загрузить спецификации
 2. **Load state** — загрузить конфигурацию
 3. **Load topology** — загрузить карты
 4. **Build dependency graph** — построить граф зависимостей
 5. **Generate code** — сгенерировать код
 6. **Update docs** — обновить документацию при изменениях
-



CHECK CODEGEN RULES

MUST (Обязательно):

- ☒ Codegen MUST читать SUMMARY_DOCS как primary source
 - ☒ Codegen MUST предпочитать canonical contracts over legacy notes
 - ☒ Codegen MUST использовать contracts из SUMMARY_DOCS/contracts/
 - ☒ Codegen MUST проверять state файлы из SUMMARY_DOCS/state/
-

- ☒ Codegen MUST обновлять документацию при изменении кода
- ☒ Codegen MUST записывать новую документацию в SUMMARY_DOCS

MUST NOT (Запрещено):

- ☒ Codegen MUST NOT использовать legacy документы вне SUMMARY_DOCS
- ☒ Codegen MUST NOT игнорировать canonical contracts
- ☒ Codegen MUST NOT создавать код в конфликт с contracts
- ☒ Codegen MUST NOT обновлять legacy документы
- ☒ Codegen MUST NOT создавать documentation вне SUMMARY_DOCS

CONFLICT RESOLUTION

Если есть конфликт между источниками:

Конфликт	Решение
Legacy vs SUMMARY_DOCS	SUMMARY_DOCS wins
Code vs Contract	Contract wins (update code)
Generated vs Canonical	Canonical wins
Old state vs New state	New state wins

Priority order для codegen:

1. **SUMMARY_DOCS contracts** (highest priority)
2. **SUMMARY_DOCS state files**
3. **SUMMARY_DOCS topology**
4. **Generated documentation**
5. **Legacy documents** (lowest priority, deprecated)

CODEGEN CONTEXT

Обязательный context для codegen:

```
{
  "entry_point": "SUMMARY_DOCS/playbooks/codegen-playbook.md",
  "contracts": [
    "SUMMARY_DOCS/contracts/project-contracts/",
    "SUMMARY_DOCS/contracts/node-contracts/"
  ],
  "state": [
    "SUMMARY_DOCS/state/node-tree.json",
    "SUMMARY_DOCS/state/doc-state.json"
  ],
  "topology": [
```

```

    "SUMMARY_DOCS/topology/NETWORK_MAP.md",
    "SUMMARY_DOCS/topology/DEPLOYMENT_MAP.md"
  ],
  "architecture": [
    "SUMMARY_DOCS/architecture/system-overview.md",
    "SUMMARY_DOCS/architecture/repo-structure.md"
  ]
}

```

Оptionальный context (по запросу):

```

{
  "migrations": "SUMMARY_DOCS/migrations/",
  "audits": "SUMMARY_DOCS/audits/",
  "playbooks": "SUMMARY_DOCS/playbooks/",
  "appendix": "SUMMARY_DOCS/appendix/"
}

```

CODEGEN DOCUMENTATION UPDATES

При генерации кода AI должен:

1. **Check contracts** — проверить спецификации
2. **Generate code** — сгенерировать код
3. **Verify compliance** — проверить соответствие contracts
4. **Update docs** — обновить документацию если код изменился
5. **Update MANIFEST** — добавить новые документы
6. **Update doc-state** — обновить метаданные

Когда обновлять документацию:

Ситуация	Действие
Новый компонент	Создать MODULE_SUMMARY.md
Изменён API	Обновить contract
Новая топология	Обновить topology map
Изменена структура	Обновить architecture docs
Новая миграция	Создать migration guide

VERIFICATION

Перед commit codegen changes:

```
# 1. Проверить соответствие contracts
node scripts/verify-contracts.js

# 2. Проверить ссылки на документы
node scripts/check-doc-links.js

# 3. Валидировать MANIFEST
node scripts/validate-manifest.js

# 4. Обновить doc-state
node scripts/update-doc-state.js
```

Checklist для codegen commit:

- ☐ Код соответствует canonical contracts
- ☐ Документация обновлена
- ☐ MANIFEST.json актуален
- ☐ doc-state.json обновлён
- ☐ Нет ссылок на legacy документы
- ☐ Все пути canonical (SUMMARY_DOCS/)

CONTRACT-DRIVEN DEVELOPMENT

Принцип:

Contract → Code → Verification → Documentation

1. **Contract first** — сначала спецификация
2. **Code generation** — генерация по contract
3. **Verification** — проверка соответствия
4. **Documentation update** — обновление docs

Пример workflow:

```
1. Прочитать NodeTreeContract.md
2. Сгенерировать код узла
3. Проверить соответствие contract
4. Обновить NODE_SUMMARY.md
5. Обновить MANIFEST.json
6. Commit changes
```

DOCUMENT CATEGORIES FOR CODEGEN

Категории и их использование:

Категория	Codegen Use	Priority
contracts/	Primary source	● Critical
state/	Configuration	● Critical
topology/	Architecture	○ High
architecture/	Structure	○ High
summary/	Reference	● Medium
migrations/	History	● Medium
playbooks/	Instructions	● Medium
audits/	Quality	○ Low
appendix/	Reference	○ Low

☑ КРИТЕРИИ ПРИЁМКИ

Codegen считается соответствующим политике если:

1. ☑ Использует SUMMARY_DOCS как primary source
2. ☑ Предпочитает canonical contracts
3. ☑ Обновляет документацию при изменениях
4. ☑ Записывает новую документацию в SUMMARY_DOCS
5. ☑ Обновляет MANIFEST.json и doc-state.json
6. ☑ Не создаёт competing sources of truth
7. ☑ Все ссылки на canonical paths

Создано: 2026-06-13

Версия: 1.0.0

Статус: Active Policy

Автор: Koda (NLP-Core-Team)

🔗 **Baloo - Проверни общение!**